SIGHT.SERIALIZATION · SPECIFICATION

# SBDL Text Format Specification

FORMAT

Sight Bundle, text encoding (`.sbdt`) and schema language (`.sgs`)

FORMAT VERSION

1

DATE

2026-10-20

COMPANION DOCUMENT

SBDL Binary Format Specification

CONTENTS


1. **Introduction**
   - 1.1 Conventions
2. **Lexical structure**
   - 2.1 Encoding
   - 2.2 Comments
   - 2.3 Identifiers and keywords
   - 2.4 Literals
   - 2.5 Punctuation
   - 2.6 Separators
3. **Type expressions**
4. **Schema language (`.sgs`)**
   - 4.1 Grammar
   - 4.2 Semantics
   - 4.3 Example
5. **Data document (`.sbdt`)**
   - 5.1 Structure
   - 5.2 Root value
   - 5.3 Exports
6. **Values**
   - 6.1 Grammar
   - 6.2 Typing rules
   - 6.3 Struct fields
   - 6.4 Type prefixes
   - 6.5 Values of type `any`
   - 6.6 Positional values
7. **References**
   - 7.1 Internal references
   - 7.2 External references
8. **Canonical output**
9. **Correspondence with the binary format**
10. **Complete example**

Appendix A. **Complete grammar**

Appendix B. **Conformance checklist for readers**

# 1. Introduction

This document defines two textual languages sharing the same lexical structure:

- the **schema language** ( `.sgs` files), used to declare schemas (§4);
- the **text data format** ( `.sbdt` files), the human-readable equivalent of the binary format (§5–§8).

The data model, the schema model, schema evolution and resource resolution are defined by the binary specification (§3, §4, §9 and §10 of that document) and are not repeated here. A text file and a binary file holding the same data are **equivalent**: conversion in either direction is lossless, except for encoding details that have no textual meaning (§9).

## 1.1 Conventions

The key words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT** and **MAY** are to be interpreted as described in RFC 2119.

A document violating a **MUST** is **invalid**; a reader **MUST** reject it with an error reporting at least the line and column of the problem, and **MUST NOT** return a partial result.

Grammar rules use EBNF: `a b` sequence, `a | b` alternative, `[a]` optional, `{a}` zero or more, `"x"` literal text. The complete grammar is given in Appendix A.

# 2. Lexical structure

## 2.1 Encoding

- Files are UTF-8, **without** byte order mark. A BOM is an error.
- Line breaks are `LF` or `CR LF`. A lone `CR` is an error.
- Whitespace is space ( `U+0020` ) and tab ( `U+0009` ). Line breaks are significant only as separators (§2.6).

## 2.2 Comments

- `//` up to the end of the line.
- `/* … */`, possibly over several lines, not nested.

Comments are equivalent to whitespace (a block comment containing a line break counts as a line break).

## 2.3 Identifiers and keywords

```
Ident = Letter { Letter | Digit }          Letter = "A".."Z" | "a".."z" | "_"
QName = Ident { "." Ident }
```

**Reserved words**, which can never be used as an identifier value (enum value, alias, anchor): `null` , `true` , `false` , `inf` , `nan` .

**Type keywords**, which cannot be used as user type names: `bool`, `u8`, `u16`, `u32`, `u64`, `s8`, `s16`, `s32`, `s64`, `f32`, `f64`, `string`, `bytes`, `list`, `map`, `ref`, `any`.

Other words (`enum`, `struct`, `abstract`, `type`) are keywords only where the grammar expects them; they remain valid field names.

## 2.4 Literals

### Integers

```
Integer   = [ "-" ] ( Decimal | Hex | Binary ) [ IntSuffix ]
Decimal   = Digit { [ "_" ] Digit }
Hex       = "0x" HexDigit { [ "_" ] HexDigit }
Binary    = "0b" BinDigit { [ "_" ] BinDigit }
IntSuffix = "u8" | "u16" | "u32" | "u64" | "s8" | "s16" | "s32" | "s64"
```

- `_` is only allowed between two digits. Leading zeros are allowed in decimal (`007`).
- Hex digits are case-insensitive. Only integer suffixes are allowed on hex and binary literals (`u` and `s` are not hex digits, so `0xFFu8` is unambiguous).

### Floats

```
Float      = [ "-" ] ( Decimal "." Decimal [ Exponent ] | Decimal Exponent | "inf" ) [
FloatSuffix ]
           | "nan" [ FloatSuffix ]
Exponent   = ( "e" | "E" ) [ "+" | "-" ] Decimal
FloatSuffix = "f32" | "f64"
```

- Digits are required on both sides of `.` (`.5` and `5.` are invalid).
- A decimal integer literal **MAY** carry a float suffix (`2f32`).
- Parsing **MUST** be correctly rounded (round-half-to-even) to the target precision.

### Strings

- Single-line: `"…"`. Escapes: `\"`, `\\`, `\n`, `\r`, `\t`, `\0`, `\u{X}` (1 to 6 hex digits, a Unicode scalar value, surrogates excluded). Any other escape is an error. Raw characters `U+0000`–`U+001F` (except tab) and line breaks are not allowed inside.
- Multi-line: `"""` immediately followed by a line break; the content ends at a line containing only whitespace followed by `"""`. The whitespace before the closing `"""` is the **indentation**: it is removed from the start of every content line, and every non-empty content line **MUST** start with it. The line break before the closing line is not part of the value. Escapes are processed as in single-line strings.

```
description: """
    First line
      indented line
```

```
    Last line
    """
```

Value: `"First line\n indented line\nLast line"`.

**Base64**

`b64"…"`: standard alphabet (RFC 4648 §4) with mandatory padding. Whitespace and line breaks inside the quotes are ignored. Only valid where a `list<u8>` ( `bytes` ) is expected.

### 2.5 Punctuation

`{ } [ ] < > ( ) : , = ? & * # @`

### 2.6 Separators

Items of a list, map or struct body, enum values and directives are separated by a comma **or** at least one line break. A trailing comma before the closing bracket is allowed. Two items on the same line without a comma are an error.

```
[1, 2, 3]          // valid
[1 2 3]            // invalid
[
  1
  2,
  3,
]                  // valid
```

Two additional line-sensitive rules:

- A **type prefix** (§6.4) and the `{` or `(` that follows it **MUST** be on the same line. So **MUST** an **explicit type** and the value that follows it (§6.5).
- A **directive** (§5.1) starts at the beginning of a line (after optional whitespace) and ends at the end of that line, except for a `{ … }` block, which may span several lines.

## 3. Type expressions

Shared by the schema language and the data format:

```
Type      = BaseType [ "?" ]
BaseType  = "bool" | "u8" | "u16" | "u32" | "u64" | "s8" | "s16" | "s32" | "s64"
          | "f32" | "f64" | "string" | "bytes" | "any"
          | "list" "<" Type ">"
          | "map" "<" Type "," Type ">"
```

```
          | "ref" "<" QName ">"
          | QName                                  // enum, struct or type alias
```

- `bytes` is a shorthand of `list<u8>`: both denote the same type, and `bytes?` is `list<u8>?`.
- A type alias denotes its target (binary spec §4.3): `Color?` with `type Color = u32` is `u32?`.

Constraints of the data model apply (binary spec §3.1): no `any?`, map keys restricted, `ref` targets are structs (not aliases).

**Type name resolution in a schema.** An unqualified name is looked up in the current schema, then in its direct imports. If it is found in more than one import, it is ambiguous and **MUST** be qualified. A qualified name is `<prefix>.<type name>`, where the prefix is an import alias (§4.2) or a schema name (`assets.Texture`).

**Type name resolution in a data document.** An unqualified name is looked up among the types of every schema of the file type table (binary spec §4.4). If it is found in more than one schema, it is ambiguous and **MUST** be qualified with the schema name.

# 4. Schema language (`.sgs`)

## 4.1 Grammar

```
SchemaFile  = "@sgs" Decimal SchemaBody
SchemaBody  = { SchemaDirective } { TypeDecl }
SchemaDirective = "@id" ( QName | String )
            | "@version" Decimal
            | "@meta" ( Ident | String ) "=" String
            | "@import" [ Ident "=" ] QName Decimal [ Hex ]
TypeDecl    = EnumDecl | StructDecl | AliasDecl

EnumDecl    = "enum" Ident ":" IntType "{" [ EnumValue { Sep EnumValue } [ "," ] ] "}"
EnumValue   = Ident [ "=" Integer ]

StructDecl  = [ "abstract" ] "struct" Ident [ ":" QName ] [ "@positional" ]
              "{" [ FieldDecl { Sep FieldDecl } [ "," ] ] "}"
AliasDecl   = "type" Ident "=" Type
FieldDecl   = Ident ":" Type [ "=" Literal ] { Annotation }
Annotation  = "@fixed" | "@alias" "(" Ident { "," Ident } ")"
```

`Sep` is a separator per §2.6. `Literal` is a scalar value literal (§6.2): boolean, number, string or enum value.

Directives follow the rules of data documents (§5.1): each starts at the beginning of a line and ends at the end of that line, and they all precede the type declarations.

| DIRECTIVE | OCCURRENCES | MEANING |
|---|---|---|
| `@sgs 1` | exactly once, **first line** of the file | schema language version (only in `.sgs` files, see §5.1 for inline schemas) |
| `@id` | exactly once | schema name; it **MUST** be a qualified name |
| `@version` | at most once | schema version (default 0) |
| `@meta` | any; keys unique | schema metadata (binary spec §4.1) |
| `@import` | any; names unique | import of another schema, with an optional local alias and an optional hash |

A `.sgs` file has the same header directives as a data document ( `@id` , `@version` , `@meta` ), so that search path indexing reads both the same way, and a compiled schema has the same header as its source.

## 4.2 Semantics

Mapping to `core.Schema` (binary spec §4.5):

| SOURCE | `CORE.SCHEMA` |
|---|---|
| `@id game` / `@version 3` | `name = "game"` , `version = 3` |
| `@import assets 2` | `SchemaRef { name: "assets", version: 2, hash }` — hash computed from the resolved import; if a hex hash is written, it **MUST** match |
| `@import tex = assets.textures 2` | same, and `tex.Texture` designates `assets.textures.Texture` in this source; the alias is not stored |
| type declarations | `types` , in declaration order |
| `enum E : u8 { A, B = 5, C }` | `EnumDef` ; a value without `=` is the previous value + 1, the first one defaults to 0 → `A = 0, B = 5, C = 6` |
| `abstract struct B : A { … }` | `StructDef { base: index of A, isAbstract: true, … }` |
| `struct V @positional { … }` | `StructDef { isPositional: true, … }` |
| `type Color = u32` | `AliasDef { name: "Color", target: TypeRef U32 }` |
| `x: Color` | `TypeRef { kind: Alias, index: index of Color }` |
| `x: bytes` | `TypeRef` of `list<u8>` ( `bytes` is not stored) |
| `x: u16 = 1` | `Field.default = 1` |
| `x: u64 @fixed` | `TypeRef.flags` bit 1 |

| SOURCE | CORE.SCHEMA |
| --- | --- |
| `x: T @alias(old, older)` | `Field.aliases = ["old", "older"]` |

Rules:

- All model rules apply (binary spec §4): unique names, acyclic inheritance and aliases, defaults restricted to scalar, string and enum types, positional structs restricted to scalar and enum fields.
- A field **MUST NOT** be both nullable and defaulted ( `T? = v` is invalid): `null` is the implicit default of a nullable field.
- `@fixed` is only valid on `u16` , `u32` , `u64` , `s16` , `s32` , `s64` (possibly through an alias), possibly nullable. Each annotation appears at most once per field or struct.
- The target of a type alias **MUST NOT** be nullable; `@fixed` is not allowed on aliases (only on fields).
- Types, aliases included, may be referenced before their declaration.
- An import alias **MUST** be an identifier, unique among aliases, and different from the schema name and from the first segment of every imported schema name.

## 4.3 Example

```
@sgs 1
@id game
@version 3
@meta author = "Arthur"
@import assets 2

enum Class : u8 { Warrior, Mage }

type Color = u32                        // RGBA
type Palette = list<Color>

struct Vec3 @positional {
  x: f32
  y: f32
  z: f32
}

struct Item {
  name:  string
  count: u16 = 1
  tint:  Color = 0xFFFFFFFF
}

abstract struct Character {
  name:  string
  level: u8 = 1
}
```

```
struct Warrior : Character { rage: u16 }
struct Mage : Character { mana: u32 }

struct Player {
  party:      list<Character>
  inventory:  list<Item>
  stats:      map<string, f32>
  leader:     ref<Character>?
  icon:       ref<Texture>            // in place, internal or external
  hash:       u64 @fixed
  experience: s32 @alias(xp)
  position:   Vec3
  colors:     Palette
  avatar:     bytes?
  extra:      any
}
```

# 5. Data document ( `.sbdt` )

## 5.1 Structure

```
Document  = FormatDirective { Directive } RootValue { ExportDirective }
```

| DIRECTIVE | OCCURRENCES | BINARY EQUIVALENT |
|---|---|---|
| `@sbdl 1` | exactly once, **first line** of the file | magic + format version |
| `@id` (QName \| String) | at most once | header `id` (default `""`) |
| `@version` Decimal | at most once | header `version` (default 0) |
| `@meta` (Ident \| String) `=` String | any; keys unique | metadata |
| `@schema` QName Decimal [ Hex ] | one or more; names unique | schema mode Reference: the file schemas, in order (name, version, optional hash) |
| `@schema` `{` SchemaBody `}` | one or more | schema mode Inline; each block contains a schema in the schema language, without the `@sgs` directive |
| `@dep` Ident `=` (QName \| String) Decimal | any; aliases and ids unique | dependency list, in order |
| `@export` (Ident \| String) `=` Value | any, **after** the root | exports, in order |

- `@sbdl` **MUST** be the very first characters of the file (no comment or blank line before it). The format version **MUST** be `1`.
- Directives other than `@export` **MUST** appear before the root value, in any order. A reference `@schema` and inline `@schema` blocks **MUST NOT** be mixed. No `@schema` at all means schema mode None.
- The file schemas (binary spec §4.4) are the schemas of the `@schema` directives, in order. Several schemas are allowed, e.g. a base schema and an extension declaring derived types.
- With inline schemas, imports not provided as an inline block are resolved externally (binary spec §10).
- With a reference `@schema`, the hash is verified when present. Writers **SHOULD** omit it.
- A `@dep` **alias** is local to the file and is only used by external references (§7.2). The dependency **id** is what is stored in the binary header.

Search path indexing (binary spec §10.3) reads `.sbdt` files up to the first token that is not a header directive.

### 5.2 Root value

The root value is a struct value with a **mandatory type prefix** (`Player { … }`), since schemas do not declare a root type. The type **MUST** be a non-abstract struct.

### 5.3 Exports

`@export name = value`, where `value` is either `*anchor` (§7.1) or a struct value with a mandatory type prefix, optionally anchored. Export names **MUST** be unique.

## 6. Values

### 6.1 Grammar

```
Value      = "null"
           | Scalar
           | List
           | Braced
           | Positional
           | Anchor ( Braced | Positional )
           | Reference
           | ExtRef
           | TypedValue                          // only where the type is `any`, §6.5

Scalar     = "true" | "false" | Integer | Float | String | MultiString | Base64 | EnumValue
EnumValue  = QName                               // not followed by a value on the same line
List       = "[" [ Value { Sep Value } [ "," ] ] "]"
Braced     = [ QName ] "{" [ Entry { Sep Entry } [ "," ] ] "}"
Positional = [ QName ] "(" [ Value { Sep Value } [ "," ] ] ")"
Entry      = Key ":" Value
Key        = Ident | String | Integer | "true" | "false"
Anchor     = "&" ( Ident | Decimal )
```

```
Reference  = "*" ( Ident | Decimal )
ExtRef     = "<" Ident [ "#" ( Ident | String ) ] ">"
```

A name (QName) followed on the same line by `{` or `(` is a type prefix; followed on the same line by another value, it is the explicit type of a TypedValue ( `Color 255` , `bytes b64"AQ=="` ); otherwise it is an EnumValue (or a reserved word).

The grammar is unambiguous without a schema: a document can always be parsed into a syntax tree. The schema is required to **type** the tree: it decides whether `12` is a `u8` or an `f32` , whether `{ … }` is a struct or a map, and whether `Mage` is an enum value.

## 6.2 Typing rules

| EXPECTED TYPE | ACCEPTED SYNTAX |
|---|---|
| `bool` | `true` , `false` |
| integer types | Integer literal within the range of the type; a suffix, if any, **MUST** equal the type |
| `f32` , `f64` | Float or decimal Integer literal; a suffix, if any, **MUST** equal the type |
| `string` | String or MultiString |
| enum `E` | value name of `E` ( `Mage` ) or qualified with the enum name ( `Class.Mage` , or `game.Class.Mage` ) |
| `list<u8>` ( `bytes` ) | List of integers or Base64 |
| `list<T>` | List |
| `map<K, V>` | Braced **without** type prefix; keys typed as `K` : String for `string` , Integer for integers, `true` / `false` for `bool` , Ident for enums |
| struct `T` | Braced with Ident keys (field names), or Positional (§6.6); type prefix rules in §6.4 |
| type alias | any syntax accepted for its target |
| `T?` | `null` or any syntax accepted for `T` |
| `ref<T>` | struct value (in place), `*name` (§7.1) or `<alias…>` (§7.2), chosen value by value |
| `any` | §6.5 |

A literal out of range for its type, or a syntax not accepted for the expected type, is an error. Integer literals are exact: `1.0` is not accepted for an integer type.

## 6.3 Struct fields

- Fields **MAY** appear in any order. A field **MUST NOT** appear twice.

- A field name that is not a field of the concrete type (including inherited fields) is an error. Aliases are **not** accepted in text: a text document is always written with the schema it references.
- A missing field takes `null` if nullable, its default if it has one, and is an error otherwise.

## 6.4 Type prefixes

A struct value **MAY** be preceded by its type name, on the same line as `{` or `(`:

- **mandatory** for the root value, exports, struct values in `any`, and when the concrete type differs from the declared type (`Warrior { … }` in a `list<Character>`);
- **optional** otherwise; if present, it **MUST** be the declared type or a type deriving from it. If the declared type is abstract, a prefix is always mandatory.

The name of a type alias whose target is a struct is accepted as a prefix, and designates that struct. Writers always use the struct name.

## 6.5 Values of type `any`

The value carries its own type:

| SYNTAX | TYPE |
|---|---|
| `null` | Null |
| `true`, `false` | `bool` |
| Integer without suffix / with suffix | `s64` / the suffix type |
| Float without suffix / with suffix | `f64` / the suffix type |
| String, MultiString | `string` |
| Base64 | `list<u8>` |
| `E.Value` (qualified enum value) | enum `E` |
| `T { … }` or `T ( … )` (mandatory prefix) | struct `T` |
| `[ … ]` | `list<any>` |
| `{ … }` with String keys | `map<string, any>` |
| `*name`, or `&name T { … }` | `ref<T>`, `T` being the concrete type of the target |
| TypedValue: `Type Value` | the explicit `Type` |

```
TypedValue = Type Value      // e.g. list<u8> [1, 2], map<s32, f32> { 1: 0.5 },
ref<assets.Texture> <tex>, Color 255
```

An explicit type is required for any other type, e.g. `list<s32> [1, 2]`, `map<Class, u8> { Mage: 1 }` or `ref<Texture> <tex>`. An unqualified enum value or an unprefixed struct is an error in `any`.

The explicit type **MAY** be or contain type aliases (`Color 255`, `list<Color> [1, 2]`), and enum values **MAY** be qualified with an alias of their enum. The value is typed with the resolved type, which is the type of the `any` value: aliases are not kept (binary spec §7.9). `Color 255` and `255u32` are the same value.

### 6.6 Positional values

A struct value **MAY** be written in positional form: its field values, in field sequence order (binary spec §4.2), between parentheses.

```
position: (1.5, 0.0, -3.0)          // Vec3 { x: 1.5, y: 0.0, z: -3.0 }
path:     [(0, 0, 0), (1, 0, 0)]
extra:    Vec3 (1, 2, 3)            // type prefix, as for braced values
```

- Accepted for any struct whose field sequence is non-empty and only contains non-nullable `bool`, integer, float, `string` or enum fields (possibly through aliases), whether or not it is declared `@positional`. Using it for another struct is an error.
- Every field **MUST** be given, defaulted ones included: the number of values **MUST** equal the number of fields.
- Each value is typed as its field (§6.2). Type prefix rules are those of braced values (§6.4); an anchor **MAY** precede a positional value at a reference position.
- A positional value is a struct, never a map.

## 7. References

### 7.1 Internal references

- `&name` (anchor) is placed before an object (braced or positional) at a `ref<T>` position, an export, or an `any` value; the object becomes referenceable.
- `*name` refers to an anchored object. The concrete type of the target **MUST** be `T` or derive from `T`.
- Anchor names are identifiers or decimal numbers, local to the document, and **MUST** be unique.
- References **MAY** precede their anchor (forward references); every reference **MUST** resolve once the document is parsed.
- An anchor on a value that is not at a `ref<T>` position, an export or an `any` value is an error.
- In `any`, an anchored struct (`&a T { … }`) is a `ref<T>`; an unanchored struct (`T { … }`) is a struct value.

The position of the anchor in the text has no meaning: it does not need to match the position where a binary writer emits the object inline.

### 7.2 External references

At a `ref<T>` position, or in `any` with an explicit `ref<T>` type:

- `<alias>`: root value of the dependency declared with `@dep alias = …`.

- `<alias#name>` or `<alias#"any name">`: export `name` of that dependency.
- An undeclared alias is an error.

The three forms can be mixed at the same position, for instance in a `list<ref<Texture>>`:

```
textures: [
  &1 { path: "local.png" }          // in place, anchored
  *1                                // internal: same object
  <tex#hero>                        // external: export "hero" of the dependency
]
```

## 8. Canonical output

Writers **MUST** produce the canonical form defined here, so that equal data gives identical text (stable diffs).

### Layout

- `LF` line breaks, final line break, no trailing whitespace, indentation of 2 spaces per level (configurable).
- Directives in this order: `@sbdl 1`, `@id` (if non-empty; qualified name form if valid, string otherwise), `@version` (if not 0), `@meta` (in order), `@schema` (one per file schema; in Inline mode, one block per schema of the closure), `@dep` (in order); then one blank line, the root value, and, if any, one blank line followed by the `@export` directives.
- Schema sources: `@sgs 1`, `@id`, `@version` (if not 0), `@meta` (in order), `@import` (in order, without alias nor hash); then each type declaration preceded by one blank line, fields one per line, enum values one per line with explicit values, `@positional` after the base of positional structs, aliases as `type Name = Type`, `bytes` for `list<u8>`, type names qualified by schema name when they belong to an import, and type references written through their alias when they have one. Inside an inline `@schema` block, the same form without `@sgs` and indented by one level.
- A list, map or struct is written on a **single line** if its whole representation fits within the maximum width (default 80 columns, measured from the start of the line) and contains no multi-line string: `[1, 2, 3]`, `{ name: "Potion", count: 3 }` (one space inside braces, none inside brackets). Otherwise it is written with one item per line, without commas.
- Empty containers: `[]` and `{}`.

### Values

- Struct fields in field sequence order (binary spec §4.2); fields equal to their default and null nullable fields are omitted (unless configured otherwise).
- Values of positional structs ( `isPositional` ) in positional form (§6.6), always on a single line, every field included, strings in single-line form; values of other structs always braced.
- Type prefixes only where mandatory (§6.4).
- Explicit types of `any` values with resolved types: no aliases, `list<u8>` written `bytes`.
- Map entries in their order; enum values unqualified unless in `any`.
- Integers in decimal, without suffix outside `any`.

- Floats: the shortest decimal representation that parses back to the same value (bitwise), with `.0` appended if it has neither `.` nor exponent; exponent notation (`1.5e-7`, `1e21`) when the decimal exponent is lower than −6 or greater than 20; `inf`, `-inf`, `nan`. Implementations **MUST NOT** rely on the .NET Framework `"R"` format, which is not round-trip safe.
- Strings: single-line with the minimal escapes (`\"`, `\\`, `\n`, `\r`, `\t`, `\0`, `\u{…}` for other control characters); multi-line form when the string contains a line break and the value is written on its own line.
- `list<u8>`: Base64.
- Anchors: only on objects referenced more than once or exported, named `&1`, `&2`… in the order of first appearance; the object is written inline at its first appearance in field sequence order.
- Dependency aliases: the last segment of the dependency id if it is an identifier unique among aliases, otherwise `d1`, `d2`… by position.

## 9. Correspondence with the binary format

| BINARY FEATURE | TEXT |
|---|---|
| Header fields, metadata, dependencies | directives (§5.1) |
| Schema modes Inline / Reference / None | `@schema { … }` blocks / `@schema name version` lines / no `@schema` |
| Exports | `@export` |
| Polymorphism tag | type prefix |
| `ref` object ids | anchors |
| `any` type descriptor | literal type, suffix or explicit type |
| `StructDef.isPositional` | positional form of the struct values (§6.6) |
| Type aliases | names in schemas only; data values have the resolved type |
| Varints, fixed encoding, presence bitmap, string deduplication, packed bools | no equivalent (encoding details) |
| Compression | no equivalent; the whole file **MAY** be compressed externally, but it is then no longer indexable by search paths |
| Schema evolution (aliases, removed fields) | not applicable to text documents (§6.3); applies after conversion to binary |

Text → binary conversion uses the writer options for `dedupMax`, compression and schema mode. Binary → text conversion of a file whose schema mode is Inline produces inline `@schema` blocks, rendered in the canonical schema language form.

## 10. Complete example

```
@sbdl 1
@id "level-01"
@version 3
@meta author = "Arthur"
@schema game 3
@dep tex = assets.textures 2
@dep items = game.items 1

Player {
  party: [
    Warrior { name: "Conan", rage: 12 }
    Mage { name: "Merlin", level: 7, mana: 300 }
  ]
  inventory: [
    { name: "Potion", count: 3 }
    { name: "Torch" }
  ]
  stats: { "str": 12.5, "dex": 8.0 }
  leader: &1 Warrior { name: "Conan", rage: 12 }
  icon: <tex>
  hash: 11473521213003792385
  experience: 1200
  position: (1.5, 0.0, -3.0)
  colors: [4278190335, 255]
  extra: b64"AQID"
}

@export captain = *1
```

## Appendix A. Complete grammar

```
(* Lexical *)
Letter       = "A".."Z" | "a".."z" | "_"
Digit        = "0".."9"
HexDigit     = Digit | "a".."f" | "A".."F"
BinDigit     = "0" | "1"
Ident        = Letter { Letter | Digit }
QName        = Ident { "." Ident }
Decimal      = Digit { [ "_" ] Digit }
Hex          = "0x" HexDigit { [ "_" ] HexDigit }
Binary       = "0b" BinDigit { [ "_" ] BinDigit }
IntSuffix    = "u8" | "u16" | "u32" | "u64" | "s8" | "s16" | "s32" | "s64"
```

```
FloatSuffix  = "f32" | "f64"
Integer      = [ "-" ] ( Decimal | Hex | Binary ) [ IntSuffix ]
Float        = [ "-" ] ( Decimal "." Decimal [ Exponent ] | Decimal Exponent | "inf" ) [
FloatSuffix ]
             | Decimal FloatSuffix
             | "nan" [ FloatSuffix ]
Exponent     = ( "e" | "E" ) [ "+" | "-" ] Decimal
String       = '"' { Char | Escape } '"'
MultiString  = '"""' LineBreak { Line } Indent '"""'
Base64       = 'b64"' { Base64Char | Whitespace | LineBreak } '"'
Escape       = "\" ( '"' | "\" | "n" | "r" | "t" | "0" | "u{" HexDigit { HexDigit } "}" )
Sep          = "," | LineBreak { LineBreak }

(* Types *)
IntType      = "u8" | "u16" | "u32" | "u64" | "s8" | "s16" | "s32" | "s64"
Type         = BaseType [ "?" ]
BaseType     = "bool" | IntType | "f32" | "f64" | "string" | "bytes" | "any"
             | "list" "<" Type ">" | "map" "<" Type "," Type ">"
             | "ref" "<" QName ">" | QName

(* Schema language *)
SchemaFile   = "@sgs" Decimal LineBreak SchemaBody
SchemaBody   = { SchemaDirective LineBreak } { TypeDecl }
SchemaDirective = "@id" ( QName | String )
             | "@version" Decimal
             | "@meta" ( Ident | String ) "=" String
             | "@import" [ Ident "=" ] QName Decimal [ Hex ]
TypeDecl     = EnumDecl | StructDecl | AliasDecl
EnumDecl     = "enum" Ident ":" IntType "{" [ EnumValueDecl { Sep EnumValueDecl } [ "," ] ] "}"
EnumValueDecl = Ident [ "=" Integer ]
StructDecl   = [ "abstract" ] "struct" Ident [ ":" QName ] [ "@positional" ] "{" [ FieldDecl {
Sep FieldDecl } [ "," ] ] "}"
AliasDecl    = "type" Ident "=" Type
FieldDecl    = Ident ":" Type [ "=" Scalar ] { Annotation }
Annotation   = "@fixed" | "@alias" "(" Ident { "," Ident } ")"

(* Data document *)
Document     = FormatDirective { LineBreak } { Directive LineBreak } Braced { LineBreak } {
ExportDirective LineBreak }
FormatDirective = "@sbdl" Decimal
Directive    = "@id" ( QName | String )
             | "@version" Decimal
             | "@meta" ( Ident | String ) "=" String
             | "@schema" QName Decimal [ Hex ]
             | "@schema" "{" SchemaBody "}"
             | "@dep" Ident "=" ( QName | String ) Decimal
ExportDirective = "@export" ( Ident | String ) "=" ( Reference | [ Anchor ] ( Braced |
Positional ) )
```

```
(* Values *)
Value        = "null" | Scalar | List | Braced | Positional | Anchor ( Braced | Positional ) |
Reference | ExtRef | TypedValue
Scalar       = "true" | "false" | Integer | Float | String | MultiString | Base64 | QName
List         = "[" [ Value { Sep Value } [ "," ] ] "]"
Braced       = [ QName ] "{" [ Entry { Sep Entry } [ "," ] ] "}"
Positional   = [ QName ] "(" [ Value { Sep Value } [ "," ] ] ")"
Entry        = Key ":" Value
Key          = Ident | String | Integer | "true" | "false"
Anchor       = "&" ( Ident | Decimal )
Reference    = "*" ( Ident | Decimal )
ExtRef       = "<" Ident [ "#" ( Ident | String ) ] ">"
TypedValue   = Type Value
```

## Appendix B. Conformance checklist for readers

A reader **MUST** reject: missing or misplaced `@sbdl` or `@sgs`; missing `@id` in a schema; unsupported format version; BOM or lone `CR`; invalid escape or control character in a string; malformed multi-line string indentation; invalid Base64; items on the same line without comma; duplicate directives, metadata keys, dependency aliases or ids, export names, anchors, struct fields or map keys; unknown or missing required fields; values not accepted for the expected type or out of range; positional value for a struct that does not allow it, or with a wrong number of values; cyclic or nullable type alias; unqualified enum value or unprefixed struct in `any`; missing mandatory type prefix; unresolved reference or undeclared dependency alias; anchor at a non-reference position; schema hash mismatch.